# Introduction to Deep Learning

## Session 2: Perceptron, MLPs & Activations

March 2026

Magda Gregorová - magda.gregorova@thws.de

Technical University
of Applied Sciences
Würzburg-Schweinfurt

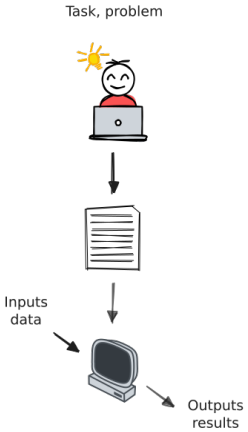
Licensed according to CC-BY-SA 4.0

# Lecture Overview

1. Supervised Learning

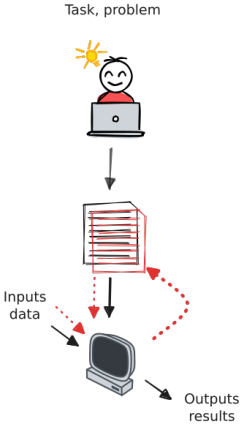
# Why Machine Learning

## Traditional Software




adapt to changing conditions
too many variations to capture
too complex to understand fully
too expensive to compute

## Machine Learning

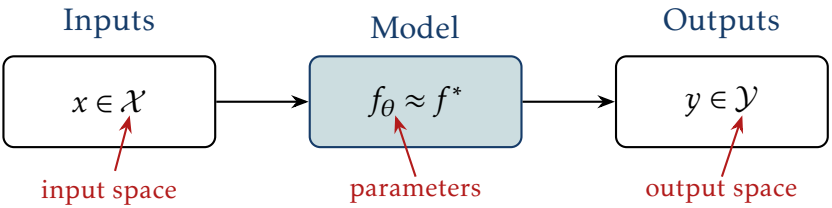


# Supervised Learning

**Setup:** data pairs $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{N}$ with $\mathbf{x} \in \mathcal{X}$, $y \in \mathcal{Y}$

**Assumption:** exists true relationship $f^* : \mathcal{X} \to \mathcal{Y}$, $y = f^*(\mathbf{x})$, but $f^*$ is never observed!

**Goal:** learn $f_\theta : \mathcal{X} \to \mathcal{Y}$ from $\mathcal{D}$ such that $f_\theta \approx f^*$ $\Rightarrow$ predict on **unseen** inputs $\mathbf{x} \notin \mathcal{D}$

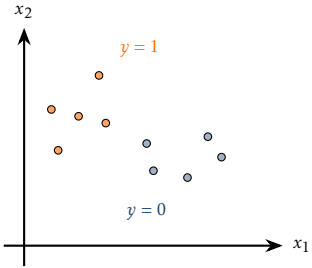



**Notation:** $f_\theta(\mathbf{x}) = f(\mathbf{x}; \theta) = \hat{y}$

# Supervised Learning - Basic Tasks

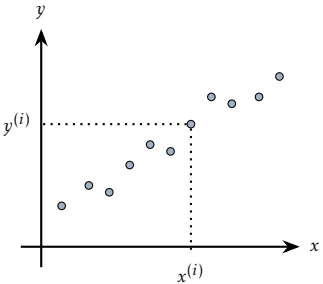
## Classification (binary)

- Output: discrete label $\mathcal{Y} = \{0, 1\}$
- Goal: find $f_\theta : \mathcal{X} \to \{0, 1\}$
- Predict category for input




## Regression

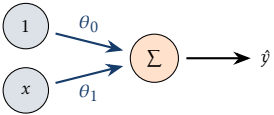
- Output: continuous value $\mathcal{Y} = \mathbb{R}$
- Goal: find $f_\theta : \mathcal{X} \to \mathbb{R}$
- Predict numerical value for input

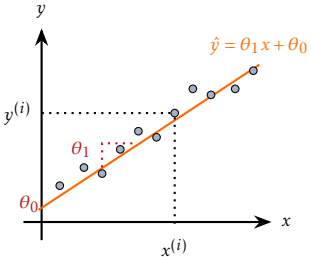


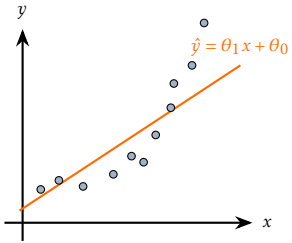
# Regression and Linear Model

**Simplest model:** linear function

$$\hat{y} = f_\theta(x) = \theta_1 x + \theta_0, \qquad \boldsymbol{\theta} = (\theta_0, \theta_1)$$




**Limitation:** real data is rarely linear

**Solution:** more expressive $f_\theta$

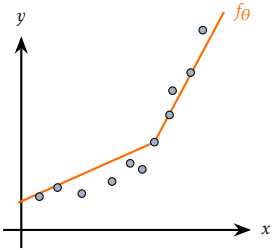
**How:** $f_\theta = \sin, \log, \sqrt{\ }, \exp, \ldots$ **?**

# Nonlinear Regression

**Idea:** piecewise linear functions

**How:** more breaks, more expressive $f_\theta$

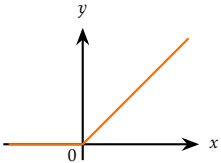
Key result: approximate any continuous func

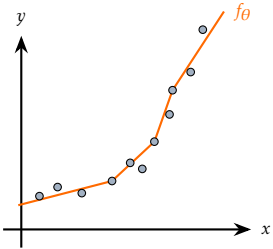



**Building block**

Rectified Linear Unit

$$\mathrm{ReLU}(x) = \max(0, x)$$

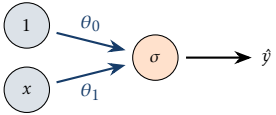


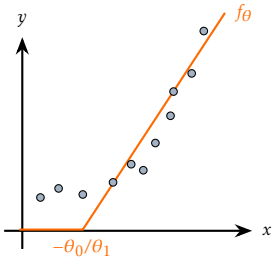



# Linear Function + ReLU

**Add nonlinearity:** RELU $\circ$ linear function

$$\hat{y} = f_\theta(x) = \sigma(\theta_1 x + \theta_0), \quad \sigma(x) = ReLU(x) = \max(0, x)$$




**Effect:** one bend at $x = -\theta_0/\theta_1$

**Limitation:** only one breakpoint - not expressive enough

**Solution:** combine multiple ReLUs

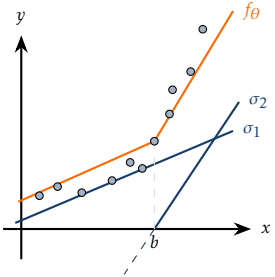
# Combining Multiple ReLUs

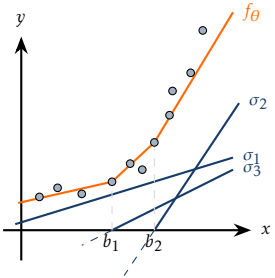
**Combine:** weighted sum of ReLU neurons

$$\hat{y} = f_\theta(x) = \theta_{10}^{(2)} + \sum_j \theta_{1j}^{(2)} \sigma\left(\theta_{j1}^{(1)} x + \theta_{j0}^{(1)}\right)$$

**Effect:** one bend per neuron

**Key insight:** more neurons $\Rightarrow$ more breakpoints $\Rightarrow$ more expressive $f_\theta$

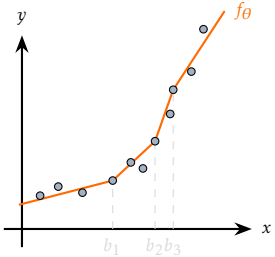


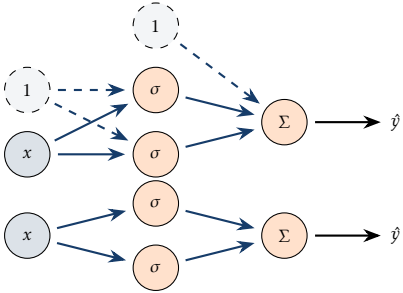



# Combining Multiple ReLUs

**Combine:** weighted sum of ReLU neurons

$$\hat{y} = f_\theta(x) = \theta_{10}^{(2)} + \sum_j \theta_{1j}^{(2)} \sigma\left(\theta_{j1}^{(1)} x + \theta_{j0}^{(1)}\right)$$




**Effect:** one bend per neuron

**Key insight:** more neurons ⇒ more breakpoints ⇒ more expressive $f_\theta$

# This is a neural network

[mlp_1hidden.tex]

- Each hidden neuron computes:

$$z_j^{(1)} = w_j x + b_j, \quad a_j^{(1)} = \mathrm{ReLU}(z_j^{(1)})$$

- Output layer combines:

$$f(x;\theta) = \sum_j v_j a_j^{(1)} + c$$

# This is a neural network

[mlp_1hidden.tex]

- Each hidden neuron computes:

$$z_j^{(1)} = w_j x + b_j, \quad a_j^{(1)} = \mathrm{ReLU}(z_j^{(1)})$$

- Output layer combines:

$$f(x;\theta) = \sum_j v_j a_j^{(1)} + c$$

- In matrix form:

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$$
$$\mathbf{a}^{(1)} = \mathrm{ReLU}(\mathbf{z}^{(1)})$$
$$f(\mathbf{x};\theta) = \mathbf{W}^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)}$$

- Parameters: $\theta = \{\mathbf{W}^{(1)}, \mathbf{b}^{(1)}, \mathbf{W}^{(2)}, \mathbf{b}^{(2)}\}$

# Universal approximation

## Universal Approximation Theorem

Let $\sigma : \mathbb{R} \to \mathbb{R}$ be a non-constant, bounded, continuous activation function. Then for any continuous function $g : [0,1]^d \to \mathbb{R}$ and any $\varepsilon > 0$, there exists a 1-hidden-layer neural network $f(\cdot;\theta)$ such that:

$$\sup_{\mathbf{x}\in[0,1]^d} |f(\mathbf{x};\theta) - g(\mathbf{x})| < \varepsilon$$

### What it tells us

- MLPs are universal approximators
- Enough neurons can fit any continuous function

### What it doesn't tell us

- How many neurons are needed
- How to find the parameters $\theta$
- Whether the model will generalize

# Classification

[linear_boundary.tex]

- Output is discrete: $y \in \{0, 1\}$
- Linear classifier: decision boundary is a hyperplane

$$\hat{y} = \mathbb{1}[f(\mathbf{x}; \theta) \geq 0]$$

# Classification

[xor_boundary.tex]

- Output is discrete: $y \in \{0, 1\}$
- Linear classifier: decision boundary is a hyperplane

$$\hat{y} = \mathbb{1}[f(\mathbf{x}; \theta) \geq 0]$$

- Not all problems are linearly separable
  - XOR: four points, no linear boundary works

# Classification

[xor_boundary.tex]

- Output is discrete: $y \in \{0, 1\}$
- Linear classifier: decision boundary is a hyperplane

$$\hat{y} = \mathbb{1}[f(\mathbf{x}; \theta) \geq 0]$$

- Not all problems are linearly separable
  - XOR: four points, no linear boundary works
- Neural network with ReLU: non-linear decision boundary
- Same architecture as for regression

# Activation functions

[activation_functions.tex]

## What makes a good activation?

- Non-linear — otherwise depth buys nothing
- Differentiable — needed for backpropagation
- Computationally cheap

## Common choices

- **Sigmoid**: $\sigma(z) = \frac{1}{1+e^{-z}} \in (0,1)$
- **Tanh**: $\tanh(z) \in (-1,1)$
- **ReLU**: $\max(0,z)$ — sparse, fast, but dying neurons

## Dying ReLU problem

- Neurons with $z < 0$ output 0 and have zero gradient
- Can become permanently inactive

# Notation summary

| Symbol | Meaning | Dimensions |
|---|---|---|
| $\mathbf{x}$ | input | $\mathbb{R}^d$ |
| $y$ | target output | $\mathbb{R}$ or $\{0, 1\}$ |
| $\theta$ | all parameters | — |
| $\mathbf{W}^{(l)}$ | weight matrix, layer $l$ | $\mathbb{R}^{d_l \times d_{l-1}}$ |
| $\mathbf{b}^{(l)}$ | bias vector, layer $l$ | $\mathbb{R}^{d_l}$ |
| $\mathbf{z}^{(l)}$ | pre-activation, layer $l$ | $\mathbb{R}^{d_l}$ |
| $\mathbf{a}^{(l)}$ | post-activation, layer $l$ | $\mathbb{R}^{d_l}$ |
| $\sigma$ | activation function | $\mathbb{R} \to \mathbb{R}$ |
| $L$ | number of hidden layers | — |

**Forward pass:** $\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \quad \mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)}) \quad \mathbf{a}^{(0)} = \mathbf{x}$